

Skill-17:

```
sahasrasagi@Sahasra: ~/OS_L × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 20
#define MAX_COMMAND 200

typedef struct
{
    int job_id;
    pid_t pid;
    char command[MAX_COMMAND];
    int running;
    int stopped;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            int status;

            pid_t result = waitpid(
                jobs[i].pid,
                &status,
                WNOHANG | WUNTRACED
            );

            if (result > 0)
            {
                if (WIFSTOPPED(status))
                {
                    jobs[i].stopped = 1;
                }
                else if (WIFEXITED(status) ||
                    WIFSIGNALED(status))
                {
                    jobs[i].running = 0;
                    jobs[i].stopped = 0;
                }
            }
        }
    }
}

void add_job(pid_t pid, char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table full\n");
        return;
    }

    jobs[job_count].job_id = next_job_id++;
    jobs[job_count].pid = pid;
    strcpy(jobs[job_count].command, command);
    jobs[job_count].running = 1;
    jobs[job_count].stopped = 0;

    printf("[%d] %d %s\n",
        jobs[job_count].job_id,
        pid,
        command);

    job_count++;
}
```

```

int main()
{
    char input[MAX_COMMAND];

    printf("==== Skill 17 =====\n");

    while (1)
    {
        update_jobs();

        printf("\nmyshell> ");

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "jobs") == 0)
        {
            list_jobs();
        }
        else if (strcmp(input, "fg ", 3) == 0)
        {
            int job_id = atoi(input + 3);
            bring_to_foreground(job_id);
        }
        else if (strcmp(input, "exit") == 0)
        {
            break;
        }
        else
        {
            int length = strlen(input);

            if (length > 0 &&
                input[length - 1] == '&')
            {
                input[length - 1] = '\0';

                while (strlen(input) > 0 &&
                    input[strlen(input) - 1] == ' ')
                {
                    input[strlen(input) - 1] = '\0';
                }

                execute_background(input);
            }
            else
            {
                printf("Use '&' for background jobs\n");
            }
        }
    }

    return 0;
}

```

```

sahasrasagi@sahasra: ~/OS_Lab/skill$ gcc skill_17.c -o skill_17
sahasrasagi@sahasra:~/OS_Lab/skill$ ./skill_17
==== Skill 17 =====

myshell> sleep 20 &
[1] 1079 sleep 20

myshell> sleep 30
Use '&' for background jobs

myshell> sleep 30 &
[2] 1080 sleep 30

myshell> jobs

Active Jobs:
[1] Running  PID=1079  sleep 20
[2] Running  PID=1080  sleep 30

myshell> exit
Use '&' for background jobs

myshell> |

```

Skill - 18:

```
sahasrasagi@Sahasra: ~/OS_L x + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

#define MAX_JOBS 10
#define MAX_COMMAND 100

typedef struct
{
    int job_id;
    pid_t pid;
    char command[MAX_COMMAND];
    int stopped;
    int running;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

void add_job(pid_t pid, char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table full\n");
        return;
    }

    jobs[job_count].job_id = next_job_id++;
    jobs[job_count].pid = pid;
    strcpy(jobs[job_count].command, command);
    jobs[job_count].stopped = 0;
    jobs[job_count].running = 1;

    printf("[%d] PID=%d started\n",
           jobs[job_count].job_id,
           pid);

    job_count++;
}

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            int status;

            pid_t result = waitpid(
                jobs[i].pid,
                &status,
                WNOHANG | WUNTRACED
            );

            if (result > 0)
            {
                if (WIFSTOPPED(status))
                {
                    jobs[i].stopped = 1;
                    printf("\n[%d] Stopped\n",
                           jobs[i].job_id);
                }
                else if (WIFEXITED(status) ||
                         WIFSIGNALED(status))
                {
                    jobs[i].running = 0;
                    jobs[i].stopped = 0;
                }
            }
        }
    }
}
```

```

void list_jobs()
{
    update_jobs();

    printf("\nJobs:\n");

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            if (jobs[i].stopped)
            {
                printf("[%d] Stopped PID=%d %s\n",
                    jobs[i].job_id,
                    jobs[i].pid,
                    jobs[i].command);
            }
            else
            {
                printf("[%d] Running PID=%d %s\n",
                    jobs[i].job_id,
                    jobs[i].pid,
                    jobs[i].command);
            }
        }
    }
}

void resume_job(int job_id)
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].job_id == job_id &&
            jobs[i].running)
        {
            if (!jobs[i].stopped)
            {
                printf("Job [%d] is already running\n",
                    job_id);
                return;
            }

            if (kill(jobs[i].pid, SIGCONT) == -1)
            {
                perror("SIGCONT");
                return;
            }

            jobs[i].stopped = 0;

            printf("Job [%d] resumed\n", job_id);

            return;
        }
    }

    printf("Job [%d] not found\n", job_id);
}

void start_job(char *command)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

```

```

        char temp[MAX_COMMAND];
        strcpy(temp, command);

        char *token = strtok(temp, " ");

        while (token != NULL && count < 19)
        {
            args[count++] = token;
            token = strtok(NULL, " ");
        }

        args[count] = NULL;

        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }

    add_job(pid, command);
}

```

```

int main()
{
    char input[MAX_COMMAND];

    printf("==== Skill 18 =====\n");

    while (1)
    {
        update_jobs();

        printf("\nmyshell> ");

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "jobs") == 0)
        {
            list_jobs();
        }
        else if (strncmp(input, "resume ", 7) == 0)
        {
            int job_id = atoi(input + 7);
            resume_job(job_id);
        }
        else if (strcmp(input, "exit") == 0)
        {
            break;
        }
        else
        {
            int length = strlen(input);

            if (length > 0 &&
                input[length - 1] == '&')
            {
                input[length - 1] = '\0';

                while (strlen(input) > 0 &&
                    input[strlen(input) - 1] == ' ')
                {
                    input[strlen(input) - 1] = '\0';
                }

                start_job(input);
            }
            else
            {
                printf("Use '&' to start a background job\n");
            }
        }
    }

    return 0;
}

```

```

sahasrasagi@Sahasra:~/OS_Lab/skill$ gcc skill_18.c -o skill_18
sahasrasagi@Sahasra:~/OS_Lab/skill$ ./skill_18
==== Skill 18 =====

```

```

myshell> sleep 30 &
[1] PID=1118 started

myshell> kill -SIGSTOP 12345
Use '&' to start a background job

myshell> jobs

Jobs:
[1] Running  PID=1118  sleep 30

myshell> resume
Use '&' to start a background job

myshell> exit
sahasrasagi@Sahasra:~/OS_Lab/skill$ |

```

Skill -19:

```
GNU nano 8.7.1 skill_19.c
***#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

#define MAX_COMMAND 200

pid_t foreground_pid = -1;

void handle_sigint(int sig)
{
    if (foreground_pid > 0)
    {
        kill(foreground_pid, SIGINT);
    }
    else
    {
        printf("\nmyshe11> ");
        fflush(stdout);
    }
}

void execute_command(char *input)
{
    char *args[50];
    int count = 0;

    char temp[MAX_COMMAND];
    strcpy(temp, input);

    char *token = strtok(temp, " ");

    while (token != NULL && count < 49)
    {
        args[count++] = token;
        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        signal(SIGINT, SIG_DFL);

        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }

    foreground_pid = pid;

    waitpid(pid, NULL, 0);

    foreground_pid = -1;
}
```

```

int main()
{
    char input[MAX_COMMAND];

    signal(SIGINT, handle_sigint);

    printf("==== Skill 19 =====\n");

    while (1)
    {
        printf("\nmysHELL> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            break;
        }

        if (strlen(input) == 0)
        {
            continue;
        }

        execute_command(input);
    }

    return 0;
}

```

```

sahasrasagi@Sahasra:~/OS_Lab/skill$ ./skill_18
==== Skill 18 =====

mysHELL> sleep 30
Use '&' to start a background job

mysHELL> sleep 30 &
[1] PID=1226 started

mysHELL> exit
sahasrasagi@Sahasra:~/OS_Lab/skill$ |

```

Skill - 20:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

#define MAX_COMMAND 200

pid_t shell_pgid;
pid_t foreground_pgid = -1;

void handle_sigststp(int sig)
{
    if (foreground_pgid > 0)
        kill(-foreground_pgid, SIGTSTP);
}

void execute_command(char *input)
{
    char *args[50];
    int count = 0;

    char temp[MAX_COMMAND];
    strcpy(temp, input);

    char *token = strtok(temp, " ");

    while (token != NULL && count < 49)
    {
        args[count++] = token;
        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        setpgid(0, 0);

        signal(SIGTSTP, SIG_DFL);
        signal(SIGINT, SIG_DFL);

        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }

    setpgid(pid, pid);

    foreground_pgid = pid;

    tcsetpgrp(STDIN_FILENO, foreground_pgid);

    int status;

    waitpid(pid, &status, WUNTRACED);
```

```

        if (WIFSTOPPED(status))
        {
            printf("\nProcess stopped. PID: %d\n", pid);
        }

        tcsetpgrp(STDIN_FILENO, shell_pgid);

        foreground_pgid = -1;
    }
}

int main()
{
    char input[MAX_COMMAND];

    shell_pgid = getpid();

    setpgid(shell_pgid, shell_pgid);

    tcsetpgrp(STDIN_FILENO, shell_pgid);

    signal(SIGTSTP, handle_sigstp);
    signal(SIGINT, SIG_IGN);

    printf("==== Skill 20 =====\n");
    printf("PID: %d\n", shell_pgid);

    while (1)
    {
        printf("\nmyshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        execute_command(input);
    }

    return 0;
}
|

```

```

sahasrasagi@Sahasra:~/OS_Lab/skill$ nano skill_20.c
sahasrasagi@Sahasra:~/OS_Lab/skill$ gcc skill_20.c -o skill_20
sahasrasagi@Sahasra:~/OS_Lab/skill$ ./skill_20
==== Skill 20 =====
PID: 1289

myshell> sleep 30
^Z
Process stopped. PID: 1292

[1]+  Stopped                  ./skill_20
sahasrasagi@Sahasra:~/OS_Lab/skill$ S|

```